Running the Architecture & Design discovery agent: locating TARGET_WORKSPACE and how we can access the FSDKC codebase under the cloud-only GitHub constraints. Fetching FSDKC source via the GitHub API and analyzing architecture hotspots. Creating the discovery report from the FSDKC analysis and writing it to the required paths.

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk domain boundary violations (H8), shared-database coupling (H9), duplicated cross-layer logic (H10), and parallel Laravel/dev-api stacks (H11).

EXECUTIVE SUMMARY

FSDKC (Klearcom voice observability platform) spans three runtimes: a Laravel 11 REST API (`backend/`, 26 PHP files), a React 18 SPA (`frontend/src/`, 15 TS/JS files), and a parallel Express dev-api (`dev-api/`, 9 JS files). Controllers are thin by LOC (avg 65), but business workflows are split inconsistently — reachability math and IVR tree building are copy-pasted across controllers, services, and dev-api. There are zero repository classes; all persistence goes through Eloquent models directly from controllers. The dominant risks are **cross-domain coupling** (Dashboard and LegacyReport controllers query both Connect and Discovery models in one schema), **shared-database coupling** (80% of business tables accessed across domains), and **parallel-stack duplication** (Laravel + dev-api mirror the same realtime/test logic). Frontend architecture is healthier (shared `api/client.ts`, Zustand, React Query hooks) but pages still embed query/mutation wiring inline and one legacy class component remains.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	65 LOC (6 API controllers)	GOOD
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	4 controllers	GOOD
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	6 files	GOOD
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	2 Eloquent relation pairs	MODERATE
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	1 file (`LegacyDataMapper`)	MODERATE
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	100% Eloquent	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0	GOOD

H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	8	HIGH RISK
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	80% (4/5 tables)	HIGH RISK
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	75 LOC (11 components/pages)	GOOD
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	5 pages/widgets	GOOD
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	2 levels	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	1 class + 1 `.jsx`	MODERATE
H10	Duplicated Domain Logic (additional)	Copy-pasted workflow sites	0	1–3	>3	6 sites	HIGH RISK
H11	Parallel Runtime Stacks (additional)	Full duplicate API implementations	0	1	>1	2 (Laravel + dev-api)	HIGH RISK

1.4 Actions Required

Hotspot	Action	Rating	Priority
H4	Encapsulate bidirectional Eloquent relations inside context-specific repositories; expose DTOs at boundaries	MODERATE	MEDIUM
H5	Replace `extract()` in `LegacyDataMapper` and `LegacyReportController` with typed DTO mappers	MODERATE	MEDIUM
H8	Split cross-domain controllers/services; introduce ACL between Connect and Discovery contexts	HIGH RISK	CRITICAL
H9	Assign table ownership per domain; replace cross-domain Dashboard queries with read-model projections	HIGH RISK	HIGH
F5	Migrate `LegacyMonitorPoller` to function component; add Error Boundary in `App.tsx`	MODERATE	MEDIUM

H10	Extract `ReachabilityCalculator` and `IvrTreeBuilder`; delete 6 duplicate implementations	HIGH RISK	CRITICAL
H11	Deprecate or proxy `dev-api/`; designate Laravel as single API source of truth	HIGH RISK	HIGH

1.5 Expected Outcomes

- Consolidating reachability and tree-building logic into domain services eliminates KPI drift between Connect checks, legacy reports, and the dev-api simulator.
- Repository interfaces and application services make controllers thin HTTP adapters testable with mocked persistence, raising confidence for schema and MongoDB changes.
- Bounded contexts with an anti-corruption layer let Connect and Discovery evolve independently — including future extraction to separate deployables.
- Retiring the parallel dev-api stack removes dual-maintenance burden and ensures local development matches production Laravel behavior.
- Frontend domain service modules and migrated legacy components produce consistent React patterns with proper cleanup and error isolation.

Full report saved to `docs/discovery/01-architecture-design.md` (orchestration UI will convert to PDF).
Pipeline artifact copy: `agent-runs/20260716T110541_bql9tn/01-architecture-design.md`.

Analysis covered **backend** (26 PHP files), **frontend** (15 TS/JS files), and **dev-api** (9 JS files) from `shende-shweta/FSDKC` via GitHub REST API.